

# 思想划分：利用混合语言模型协同作用实现高效的设备端代理

邵晨阳 电子工程系 北京清华大学  
学 BNRist 中国 北京 shaocy24  
@mails.tsinghua.edu.cn

胡新元 量化理论与方法系 埃默里大学  
美国乔治亚州 nate.hu@emory.edu

林玉堂 清华大学 电子工程系  
中国北京 yt-lin21@mails.  
tsinghua.edu.cn

徐峰利 \* 清华大学电子工程系  
BNRist 北京，中国 fenglixu  
@tsinghua.edu.cn

## 摘要

网络内容的快速扩张使得设备端人工智能助手在帮助用户管理日益复杂的在线任务方面变得不可或缺。大型语言模型中涌现的推理能力为下一代设备端人工智能代理提供了一个有前景的路径。然而，在资源有限的本地设备上部署全规模的大型语言模型（LLMs）具有挑战性。在本文中，我们提出了“思路分解”（DoT），这是一种协作推理框架，利用本地部署的小型语言模型（SLMs）和基于云的 LLMs 之间的协同作用。DoT 利用一个任务分解器来激发言言模型固有的规划能力，将用户查询拆分为更小的子任务，从而使混合语言模型能够充分发挥各自的优势。此外，DoT 采用一个任务调度器来分析子任务的成对依赖关系并创建依赖图，促进子任务的并行推理

## 关键词

大语言模型，LLM 推理，人工智能代理，边缘 - 云协作

## ACM 参考格式：

邵晨阳、胡新元、林玉堂、徐峰利\*。2025。《思维分工：利用混合语言模型协同实现高效的设备端代理》。收录于《2025 年 ACM 万维网会议论文集（WWW '25）》，2025 年 4 月 28 日 -5 月 2 日，澳大利亚悉尼，新南威尔士州》。ACM，美国纽约，12 页。<https://doi.org/10.1145/3696410.3714765>

## 1 引言

随着网络内容的指数增长，设备上的人工智能助手已成为帮助用户应对日益复杂的在线环境的重要工具。这一趋势促使个人助手广泛普及，例如谷歌助手、苹果 Siri、亚马逊 Alexa、阿里巴巴天猫精灵和小米小 AI，它们已证明在帮助用户处理海量网络内容方面非常有效，如网页浏览、内容搜索、在线购物和旅行计划。这些由人工智能驱动的助手使网络应用能够利用人工智能技术的快速进步，提供更个性化和便捷的用户体验。在近年来大语言模型（LLM）的人工智能突破中，常识推理和上下文学习的新兴能力被广泛认为是下一代设备端助手的关键组成部分。因此，革命性地改变人工智能个人助理

## CCS 概念

- 计算方法 → 自然语言生成；
- 计算机系统组织 → 客户端 - 服务器架构。

\*通讯作者。<sup>1</sup> 代码可在以下网址获取：<https://github.com/tsinghua-b-lab/DoT>



本作品采用知识共享署名 4.0 国际许可协议授权。WWW '25，澳大利亚，新南威尔士州，悉尼 © 2025 版权所有 / 作者所有。ACM ISBN 979-8-4007-1274-6/25/04 <https://doi.org/10.1145/3696410.3714765>

然而，在本地设备上部署大型语言模型（LLM）代理面临重大挑战，因为在资源受限的设备（如智能手机和个人电脑）上运行拥有数万亿参数的 LLM 是不现实的 [41]。相反，仅依赖基于云的商业 LLM 会引发隐私风险、不可靠的连接以及高昂的经济成本的担忧 [22]。近期的研究集中在训练小规模语言模型和开发模型压缩技术 [23, 24, 40]，目标是创建可在本地设备上实际部署的参数低于 100 亿的模型，例如 Llama 3 系列 [10] 和 Phi 3 系列 [1]。然而，这种方法在训练或压缩这些模型时引入了额外的计算成本，并不可避免地导致

与全尺寸大型语言模型（LLMs）相比，性能有所下降。初步研究也探索了边缘 - 云协作的潜力，其中超出本地部署的小型语言模型（SLMs）能力的任务会被重新路由到更强大的基于云的 LLMs [5, 13]。尽管如此，SLMs 与 LLMs 之间显著的性能差距通常导致推理能力与成本之间出现次优的权衡。

为了解决这个问题，我们的工作从经济学的基本概念“劳动分工”中获得灵感，该概念认为，将复杂任务分解为更小的组成部分通常能够带来更高效的解决方案，因为这可以让协作伙伴充分发挥各自的优势。在此基础上，我们提出了一种新颖的“思想分工”（DoT）框架，通过复杂的任务分解和优化的子任务分配，充分利用本地部署的 SLM 与云端 LLM 之间的协同效应。具体来说，DoT 利用一个任务分解器，这是一种元提示，将类似“思维链”的提示方法与精心策划的任务分解示例结合起来。这种方法能够发挥语言模型的内在规划能力，使它们能够将用户查询分解为更小的子任务。我们的核心见解是，即使是复杂的用户查询，通常也包含大量可以充分处理的简单子任务。

我们在七个广泛采用的 LLM 代理基准上进行了大量实验，涵盖多种场景，包括逻辑推理、网页浏览、数学问题求解和常识推理。结果表明，我们的 DoT 框架在显著降低 LLM 成本的同时，保持了与所有基准中最优基线方法相当的推理准确性。具体而言，与最准确的基线相比，平均推理时间和 API 成本分别降低了 66.12% 和 83.57%。此外，消融研究确认了我们关键模型设计选择的有效性。此外，与未利用“分解与分配”策略的任务转交方法相比，DoT 框架在成本 - 准确性权衡上始终表现优越，其中在低预算设置下性能增益尤其显著。我们进一步展示了 DoT 在不同基准上的强泛化能力。

总而言之，我们的贡献有三方面：

- 我们提出了一种新颖的思维分工（DoT）框架，充分利用本地部署的 SLM 与基于云的 LLM 之间的协同作用，为设备端智能体提供具有成本效益的推理能力。
- 我们提出了一种自我强化的训练方法，以在无需额外人工标注的情况下提升 SLM 的任务分配准确性，同时通过可拆卸、即插即用的适配器保留 SLM 的通用推理能力。
- 我们在各种基准上进行了广泛的实验，以评估我们 DoT 框架的有效性，并发现 DoT 在保持高推理准确性的同时可以显著降低大型语言模型的成本。

## 2 相关工作

### 2.1 使用大型语言模型进行推理

近年来，大型语言模型（LLMs）取得了显著进展，模型规模的不断扩大带来了强大的新兴推理能力。[37, 39]。此外，各种提示工程技术被提出以进一步扩展 LLMs 的推理能力。链式思维（Chain-of-Thought, CoT）[38] 通过在提示中加入手工设计的分解步骤来提高推理性能，使 LLM 能够遵循逐步解决问题的过程。在此基础上，零样本链式思维（Zero-shot CoT）[18] 仅需添加“让我们一步一步思考”这句话便能实现类似效果，使 LLM 能够自动分解和执行任务。基于 CoT 的线性结构，引入了更复杂的框架，如思维树（Tree-of-Thought）[43] 和思维图（Graph-of-Thought）[2]，通过分支推理路径推动推理能力边界的扩展。然而，这些方法面临更高的资源需求和时间复杂度，并且高度依赖手工预定义的针对性步骤。

这些方法的基础是任务分解的直觉，大量实证研究已经证明了其有效性。这激励我们探索任务分解在边缘 - 云协作中的潜力，允许部分任务由基于云的模型处理，而其余部分由设备端模型处理，从而实现一个细粒度的协作框架。

### 2.2 设备端 LLM 代理

大型语言模型（LLM）的兴起彻底改变了人工智能的应用，引发了个人 AI 助手和移动自动化工具的关注。随着设备端智能代理的普及，用户期望在智能手机上获得无缝的实时 AI 支持 [11]。然而，边缘设备有限的计算和存储能力在为这些代理部署强大模型时带来了挑战。在这些限制下，边缘 - 云协作是一个切实可行的解决方案。苹果公司最新的研究 [12] 通过将高效的设备端模型 AFM-on-device 与强大的云端模型 AFM-server 结合，体现了这种协同。这种方法平衡了设备的限制与苹果 AI 功能对高性能的需求。类似地，[6] 通过将任务规划和执行分配给两个模型——Octo-planner 和

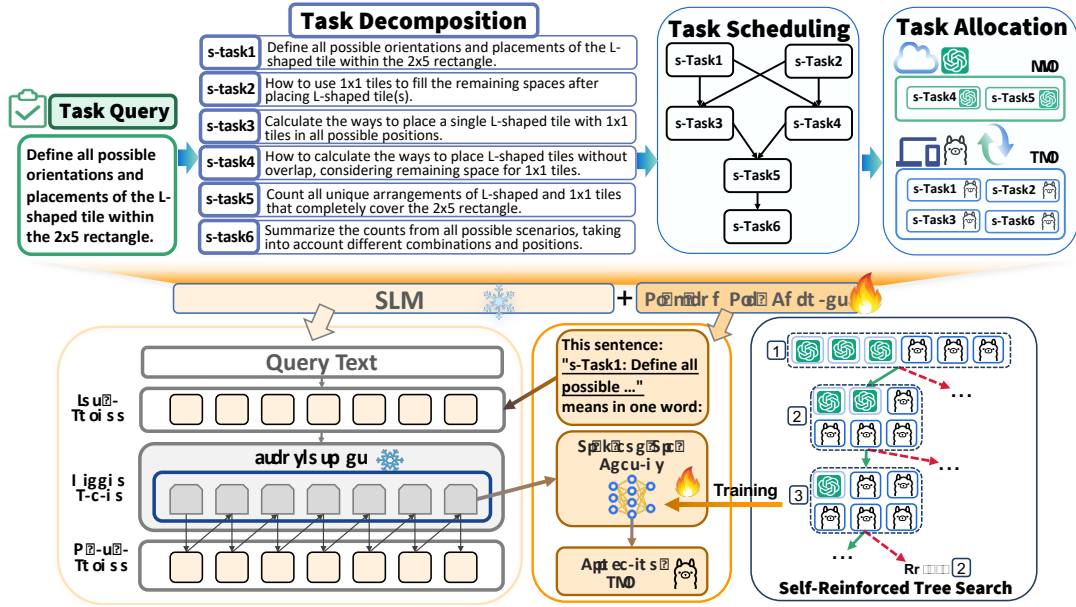


图 1：我们提出的 DoT 框架概览。

章鱼——专注于效率和适应性。这两种方法都突出了边缘与云协作的趋势，以确保强大、低延迟的人工智能体验，尽管苹果更多依赖云支持，而章鱼规划者则强调设备端优化。

### 3 预备事项

**问题定义**将本地部署的 SLM 记作  $M_D$ ，云端的 LLM 记作  $M_C$ 。用户的原始查询限于边缘模型进行任务分解和分配，而生成的子任务可以由  $M_D$  或  $M_C$  解决。整个推理任务集合表示为  $\mathcal{T} = \{T_1, T_2, \dots, T_n\}$ 。设整个任务集合的推理准确性记作  $Acc$ ，API 成本记作  $C_{Api}$ ，完成时间记作  $C_{Time}$ 。对于每个任务  $T$ ，将分解过程表示为：

$$T \rightarrow \{t^1, t^2, \dots, t^k\} \quad (1)$$

基于分解后的子任务  $t^i$ ，模型分配方案可以表示为：

$$M : t^i \mapsto \{M_D, M_C\} \quad (2)$$

它优先将简单子任务分配给设备上的 SLM，同时调用基于云的 LLM 来处理复杂子任务。

我们优化的目标是最小化模型的分配方案  $M$  与最优方案  $M^*$  之间的差异：

$$\min |M - M^*| \quad (3)$$

通过一种搜索策略导出最优方案，该策略在保持准确性的同时最大化 SLM 的使用。在优化过程中，随着分配方案逐渐接近最优解，时间成本和 API 成本下降，而仍然保持良好。

## 4 方法论

### 4.1 思维划分框架

劳动力分工是经济学中的一个重要概念，由亚当·斯密著名地提出。它指的是在任何经济系统或组织中任务的分离和分配，使参与者能够根据自身的独特能力在特定领域进行专业化。这种专业化使个人、组织和国家能够通过利用专业技能、设备和资源来优化生产力。这个概念启发我们在边缘-云协作的背景下考虑类似的方法，其中用户查询可以被精细地分解：较简单的子任务可以分配给 SLM，而更复杂的任务则分配给 LLM。这种子任务级的劳动力分工预计能够在保持推理性能的同时降低推理成本，从而实现更高效的协作。

在此直觉的基础上，我们开发了 **DoT** 边缘-云协作框架。如图 1 所示，我们的框架分为三个部分。第一部分是任务分解器，它将用户的查询拆解为几个更简单且相互独立的子任务。第二部分是任务调度器，用于确定子任务之间的成对依赖关系，并基于这些依赖关系构建任务依赖图。第三部分是即插即用 LLM 适配器，负责将每个子任务分配给合适的模型。适配器从 SLM 中提取句子嵌入并将其映射到难度系数，这些系数作为子任务模型分配的基础。重要的是，适配器的训练不需要修改 LLM 的参数，从而确保 LLM 的问答功能不受影响。一旦为每个子任务分配了合适的模型

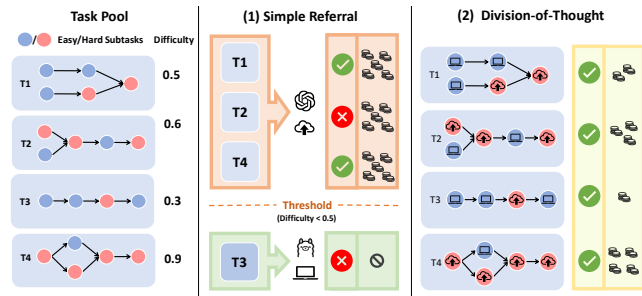


图 2: “分而配”策略的优势

清楚地表明, 与简单的推荐策略相比, DoT 在准确性和效率方面都表现出显著优势。

## 4.2 分解用户查询

正如在劳动分工中所强调的, 一个有效的分工方法是协作工作的基础。分工的粒度和准确性直接影响协作的质量和效率。以往关于模型协作的研究主要集中在查询级粒度, 并严重依赖人工任务分解策略。一个典型的例子是在 24 点游戏中的 ToT 方法, 它采用固定的两步流程: 提议生成和评估。这种僵硬的、人工设计的工作流程在不同任务之间缺乏通用性, 凸显了自动化、灵活和精细化任务分解方法的必要性。

为了解决这些挑战, 我们开发了一种基于大型语言模型 (LLMs) 强大上下文学习 (ICL) 能力的细粒度任务分解方法。我们利用一个元提示 (meta-prompt), 其中结合了类似“思路链” (chain-of-thought) 的提示与手工制作的任务分解示例, 以引出 SLMs 内在的规划能力。对于每个基准测试, 我们随机选择了 8 个样本, 手动执行逐步任务分解, 并将这些步骤纳入提示中。为了使 LLMs 能够独立解决每个子任务, 并有效利用前面任务的答案作为后续推理的参考, 我们非常重视分解子任务的独立性和清晰性, 并在提示设计中加入了针对性的提示。

## 4.3 通过依赖图进行任务调度

我们采用三步法来有效地安排所有子任务: **依赖判断**。提示大语言模型 (LLM) 代理评估子任务之间的成对依赖关系。提示如下: “请以‘子问题 A [xxx] -> 子问题 B [xxx]’的格式列出依赖关系, 表示子问题 A 必须在子问题 B 开始之前完成。”  
**图构建**。将依赖关系转换为依赖图。根据明确的定向依赖关系, 构建图是直接的。然而, 我们进一步从最终子任务节点回溯, 以计算图中每个子任务节点的深度 (类比为树结构中的高度)。处于同一深度的子任务相互独立, 可以并行推理。深度作为推理批次, 每个批次按顺序处理, 而批次内的任务

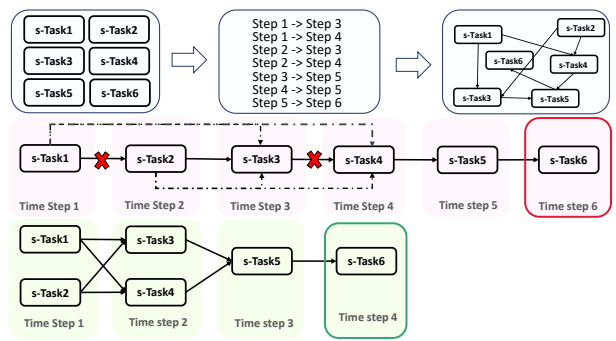


图 3: 任务调度的依赖关系图示

对于所有子任务, 我们的图结构可以在降低时间成本的同时捕捉更精确的推理关系。**图上推理**。我们从解决最浅深度批次的子任务开始, 并在同一批次内并行运行子任务。完成一个批次后, 我们继续下一个批次。在推理特定子任务时, 仅将其前置子任务的结果包含在提示中, 而不是所有已解决的任务, 这使得基于图的高效推理成为可能。

## 4.4 使用增强 SLM 的任务分配

对于分解后的子任务, 我们旨在根据任务描述评估其难度, 并为执行分配基于云端或边缘端的模型。使用大型语言模型 (LLM) 来评估难度会带来额外的推理成本, 并且常常无法准确评估子任务的复杂性, 这促使我们专门训练一个用于任务分配的模型, 该模型使用句子嵌入。句子嵌入将句子映射到固定大小的向量, 以捕捉其语义含义和上下文, 在自然语言处理中的应用非常广泛, 因为它们轻量且能够强有力地捕捉句子语义。我们可以为每个子任务的提示获取句子嵌入, 然后将该嵌入映射到相应的难度。鉴于大型语言模型已经部署在边缘端, 并且已被证明可作为有效的句子嵌入器, 我们计划利用本地部署的 SLM 生成句子嵌入。

然而, 自回归大语言模型 (LLM) 缺乏像 BERT 的 [MASK] 或 [CLS], 这样的专用标记, 而这些标记通常用于基于 Transformer 的模型中的嵌入任务。Ting Jiang 等人的研究 [15] 帮助我们克服了这一限制。基于提示的方法专门为自回归 LLM 优化, 通过指导模型生成捕捉输入句子语义的下一个词汇。具体来说, 可以为每段文本构建一个简单但有效的模板: 这个句子: “[text]” 的意思用一个词来说是: “, 其中 [text] 表示输入句子, LLM 被提示生成下一个标记, 该标记用一个词概括句子的核心意思。最后生成的标记起着关键作用, 因为我们会从模型中提取其隐藏状态并将其用作句子嵌入。

**通过即插即用适配器提升 SLM**。尽管子任务描述的长度各不相同, 句子嵌入仍保持一致的维度。为了提高效率, 我们在大型语言模型 (LLM) 的变压器模块上附加了一个多层感知器 (MLP), 用于将嵌入映射到难度分数。鉴于两者的边界

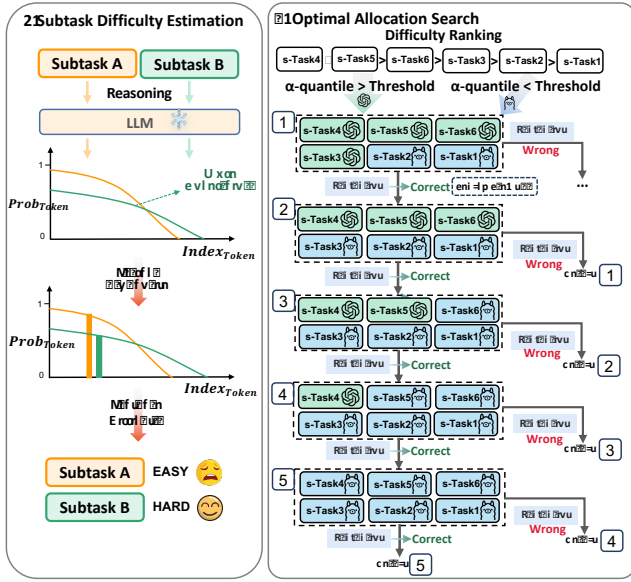


图 4：基于树搜索的数据集构建过程

在边缘侧和云端只运行一个模型的情况下，我们可以简单地将分数设置为 0（简单，适用于 SLM）或 1（复杂，适用于 LLM）。MLP 的输出会被转换为模型标识，并传递给处理当前推理步骤的代理，从而提供边缘 - 云协作的具体分配策略。此外，适配器不需要修改 SLM 的参数。它本质上作为 SLM 的一个灵活且可扩展的适配器，避免了在设备上存储两套参数的需求。

**使用  $\alpha$ -树算法创建训练数据。**我们如何构建数据集来训练我们的适配器？我们开发了一种高效的分配优化策略， $\alpha$ -Tree，它包括两个关键组件：基于  $\alpha$ -分位数令牌概率的子任务难度排名，以及由排名引导的基于树的搜索策略，使我们能够以低成本和高速生成大规模的最优分配数据集。 $\alpha$ -分位数指的是计算 LLM 在推理过程中生成的令牌概率的特定分位数。与通过求和或平均聚合概率的传统方法不同， $\alpha$ -分位数关注不确定性分布的特定部分，例如最小值（ $\alpha=0$ ）或较高的百分位数（如  $\alpha=0.8$ ），这提供了不确定性的精细测量。将输入上下文记为  $x$ ，输出令牌记为  $\hat{y}'_i$ 。 $\alpha$ -分位数值表示

$$s_{\text{quant}}(x, \alpha) \doteq \text{quantile}_{\alpha} \left( p^{(1)}(\hat{y}'_1 | x), p(\hat{y}'_2 | x, \hat{y}'_1), \dots, p(\hat{y}'_n | x, \hat{y}'_1, \dots, \hat{y}'_{n-1}) \right) \quad (4)$$

在我们的实现中，我们使用 SLM 来回答所有分解后的子问题，并记录每个标记对应的采样概率。然后我们应用一致的 alpha 值从每个概率序列中提取  $\alpha$  分位数，并对所有子任务回答的  $\alpha$  分位数值进行排序。由于任务难度较高会导致模型答案的不确定性增加，从而导致标记采样概率降低，我们反转  $\alpha$  分位数排名以获得子任务的难度顺序。

**算法 1 自增强数据集构建方法：** $\alpha$ -Tree 输入：已分解为多个子任务的任務： $T_1 = \{t_1^1, \dots, t_1^{k_1}\}$ ,  $T_2 = \{t_2^1, \dots, t_2^{k_2}\}$ , ...,  $T_n = \{t_n^1, \dots, t_n^{k_n}\}$  输出：应用于每个任务中每个子任务的细粒度设备 - 云模型分配方案 ( $m^j_i \in \{M_D, M_C\}$ ):  $\{M_1 = \{m_1^1, \dots, m_1^{k_1}\}, M_2 = \{m_2^1, \dots, m_2^{k_2}\}, \dots, M_n = \{m_n^1, \dots, m_n^{k_n}\}\}$  1: 对于任务 id  $i$  执行 2: 对每个子任务执行图上推理:  $\text{Answer} = \{a_i^1, \dots, a_i^{k_i}\}$  使用  $\text{ProbToken} = \{p_{v_i^1}^1, \dots, p_{v_i^{k_i}}^{k_i}\}$  计算每个标记序列的  $\alpha$  分位值:  $v_{\alpha\text{-quantile}} = \{v_i^1, v_i^2, \dots, v_i^{k_i}\}$  4: 设置阈值  $\theta$  以获得初始分配策略:  $M_i = \{m_i^1, \dots, m_i^{k_i}\}$  其中  $m^j_i = \begin{cases} M_D & \text{如果 } v^j_i > \theta M_C \\ M_C & \text{如果 } v^j_i < \theta \end{cases}$  5: 对任务进行推理以获得: 结果 = 真或假 6: 当 True 时执行 7: 如果结果 == 为真, 则 8: 找到分配给  $M_C$ ,  $M_{Ci} = \{m \in M_i \mid m == M_C\}$  的所有子任务, 选择具有最高  $\alpha$  分位概率的  $N$  个子任务, 并重新分配给  $M_{Ci}$

基于子任务的  $\alpha$  分位值，我们设定了一个固定的分配阈值：超过该阈值的子任务分配给 SLM，而低于阈值的子任务分配给 LLM，从而形成初始模型分配。在这一初始分配基础上，我们进行 SLM 和 LLM 的协同推理。如果推理结果正确，我们将当前由 LLM 处理的  $n$  子任务中，特别是那些最有可能被正确回答的子任务，重新分配给 SLM。相反，如果答案错误，我们将由 SLM 处理的  $n$  子任务中，特别是那些最低概率被错误回答的子任务，转移到 LLM。正如图 4 所示，搜索结构呈现树状形式。数据集构建的算法流程如算法 1 所示。

## 5 个实验

### 5.1 实验设置

**基准测试。**我们在七个开源基准测试中验证了我们框架的有效性。对于每个基准测试，我们随机选择 8 个实例进行人工标注，以用作任务分解的上下文学习示例。此外，我们

| Model            | Logical Reasoning |         |         |            |         |         | Web Browsing |         |         | Solving Math Problems |         |         |            |         |         |            |         |         | Commonsense Reasoning |         |         |
|------------------|-------------------|---------|---------|------------|---------|---------|--------------|---------|---------|-----------------------|---------|---------|------------|---------|---------|------------|---------|---------|-----------------------|---------|---------|
|                  | P3                |         |         | SCAN       |         |         | WebShop      |         |         | MATH                  |         |         | CHAMP      |         |         | DROP       |         |         | CSQA                  |         |         |
|                  | Acc               | CTime   | CAPi    | Acc        | CTime   | CAPi    | Acc          | CTime   | CAPi    | Acc                   | CTime   | CAPi    | Acc        | CTime   | CAPi    | Acc        | CTime   | CAPi    | Acc                   | CTime   | CAPi    |
| COT (GPT-4o)     | <b>42%</b>        | 35.8    | 4.45€   | <b>68%</b> | 9.21    | 2.75€   | 35%          | 30.9    | 10.65€  | 51.5%                 | 34.5    | 5.34€   | 55.5%      | 26.4    | 4.45€   | 80%        | 11.6    | 1.30€   | 80%                   | 17.0    | 3.60€   |
| TOT (GPT-4o)     | 38%               | 93.1    | 14.55€  | 52%        | 32.5    | 9.82€   | <b>36%</b>   | 62.4    | 47.34€  | <b>63%</b>            | 60.5    | 9.97€   | 57%        | 64.2    | 11.65€  | 80.5%      | 40.2    | 5.41€   | 82%                   | 98.8    | 20.50€  |
| COT (Llama 3-8B) | 5.5%              | 18.1    | N/A     | 17%        | 5.0     | N/A     | 0.0%         | 10.5    | N/A     | 10%                   | 21.1    | N/A     | 19%        | 13.1    | N/A     | 72%        | 3.8     | N/A     | 70%                   | 8.4     | N/A     |
| TOT (Llama 3-8B) | 5.5%              | 58.3    | N/A     | 13%        | 21.8    | N/A     | 1.4%         | 22.5    | N/A     | 29.5%                 | 49.0    | N/A     | 25%        | 68.1    | N/A     | 65%        | 27.8    | N/A     | 68.5%                 | 89.4    | N/A     |
| DataShunt        | 14%               | 25.1    | 2.45€   | 23.5%      | 7.6     | 1.72€   | 34%          | 30.9    | 8.35€   | 16%                   | 24.9    | 1.66€   | 34%        | 19.1    | 2.98€   | 74%        | 8.6     | 0.60€   | 73%                   | 10.4    | 1.28€   |
| DoT (ours)       | 41%               | 23.5    | 1.58€   | 63%        | 5.5     | 1.20€   | 31%          | 17.2    | 4.97€   | 59%                   | 22.6    | 1.02€   | <b>58%</b> | 16.1    | 0.84€   | <b>85%</b> | 4.9     | 0.32€   | <b>82%</b>            | 9.9     | 0.49€   |
| Improvement      | ↓2.38%            | ↓34.36% | ↓64.49% | ↓7.35%     | ↓40.28% | ↓56.36% | ↓13.89%      | ↓72.43% | ↓89.95% | ↓6.35%                | ↓62.72% | ↓89.50% | ↑1.75%     | ↓74.92% | ↓92.79% | ↑5.59%     | ↓87.81% | ↓94.09% | 0%                    | ↓89.98% | ↓97.60% |

表 1: DoT 与基线在 7 个基准测试上的表现。CTime 和 CAPi 是每项任务的平均开销，其中时间消耗以秒为单位，API 成本以美分 (¢) 计量。N/A 出现在仅使用 LLaMA 进行推理而未调用 OpenAI API 密钥的实验中。在每个基准测试中，最高的推理准确率用加粗显示。基线中具有最高 Acc 的结果用下划线标出，将用于计算最后一行的“提升”。

从每个基准中随机抽取 200 个问题来构成我们的测试集。这些基准分为四类：

- **逻辑推理**。我们选择 P3 [30] 和 SCAN[20] 进行评估。这些基准更强调逻辑，涉及诸如遍历、逆向推理和异常检测等挑战，这些都需要更高水平的逻辑连贯性和严谨性。
- **数学问题解决**。这一类别的 benchmark 标记 pri-主要涉及数学问题，包括计算、数学知识和问题解决技巧。它被广泛认为是对大型语言模型推理能力的关键测试。我们为这一类别选择了三个基准测试：MATH [14]、CHAMP [25]，和 DROP [9]。
- **常识推理**。CSQA [36] 是一个广泛使用的常识推理数据集，它对推理能力的要求较低，但需要更广泛的知识库。大型语言模型（LLM）参数规模的差异导致它们所拥有的知识系统存在差异，使得 CSQA 非常适合用于评估我们的协作推理场景。
- **网页浏览**。我们选择 WebShop [42]，它测试 LLM 作为现实场景中的交互代理的能力。它挑战 LLM 代理在网页上导航、理解复杂查询，并采取适当的行动以满足用户请求。该基准测试与 AI 助手场景高度契合，包括指令理解、查询重构和战略性信息探索。

**基线与提示方法**。鉴于边缘-云协作环境的限制，我们建立了三个基线。

- **CoT [38]**。这个基线将问题分解为子步骤，逐步解决每个子步骤。我们在整个过程中使用单一的 LLM，每个步骤只执行一次，不进行迭代。
- **ToT [43]**。ToT 为每个子步骤探索多种解决方案（M=5）。我们使用评分机制保留得分最高的 N=3 条路径，其中得分最高的路径决定最终答案。ToT 还使用单个大型语言模型进行推理。
- **DataShunt [3]**。这种方法在每个任务开始时动态选择本地 SLM 和基于云的 LLM。它提示 LLM 评估任务复杂性，从而分配一个解决模型，该模型随后在所有子步骤中一致使用。

在 CoT 和 ToT 中涉及的任务分解过程中，我们采用了在 DoT 中使用的相同提示方法，应用

每个基准有八个手工制作的示例。这些示例是从数据集中随机抽取的，并且与测试集正交。具体的提示可以在附录 B.5 中找到。

**大型语言模型的选择与部署**我们使用 GPT-4o [29] 作为基于云的 LLM，并使用 Llama 3-8B [27] 作为设备端 LLM，因为其参数规模在理论上可部署于边缘设备。Llama 3-8B 模型部署在本地 A100 GPU 上，上下文长度限制为 8192。我们的适配器参数数量仅为 13,109,249，大约是本地 Llama 参数的 1/800。

**评估**。对于评估，我们建立了三个指标：Acc, Ctime, CAPi。六个基准测试（不包括 WebShop）的结果是确定性的。例如，CSQA 的结果是单字母选项，可以直接判断正确性。类似地，对于 P3，推理输出可以传入问题函数，如果函数返回 True，则认为推理是正确的。对于 WebShop，我们使用原始基准提供的评估环境，根据约束条件平均所有购买商品满意度奖励。

## 5.2 主要结果

我们的方法与基线的比较显示在表 1 中，我们用加粗标出每个基准测试中五个基线实验的最高准确率结果，同时将相关成本用下划线标出。我们计算了我们结果相对于准确率最高的基线的相对提升。

实验结果表明，我们的方法在显著降低成本的同时，将准确度保持在可接受的下降范围内。在七个基准测试中，与最佳基线结果相比，准确度的相对变化分别为：-2.38%、-7.35%、-13.89%、-6.35%、1.75%、5.59% 和 0%。值得注意的是，在某些情况下观察到了正向提升，这表明我们的框架有潜力增强大型语言模型的推理能力。这一增强在消融研究 5.4 中有进一步讨论。同时，与具有最高准确度的基线相比，我们的方法实现了成本的大幅降低，平均时间减少 66.12%，平均 API 成本减少 83.57%，远远超过了准确度的下降。

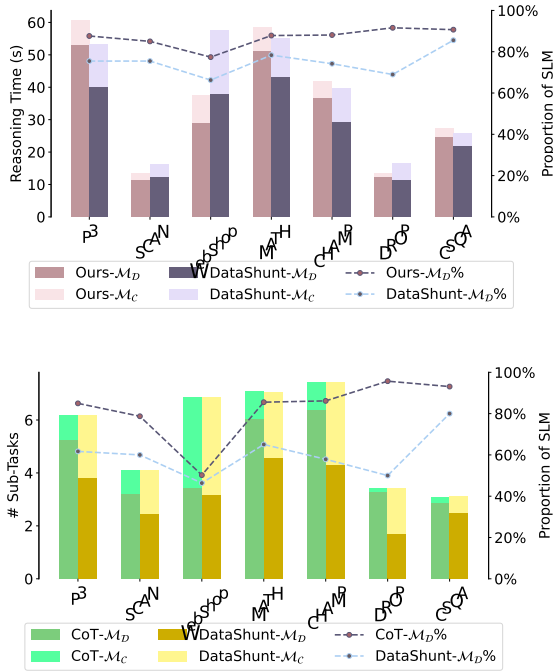
此外，在 P3 和 SCAN 基准测试中，CoT 达到了最佳性能，这表明顺序线性推理结构更适合这些任务类型。相反，在更复杂的数学推理基准测试如 MATH、CHAMP 和 WebShop 中，ToT 达到了最高的推理准确率，展示了多提议策略的有效性。然而，

| Methods              | SLM Ratio | SR     | # Evaluation | API Cost |
|----------------------|-----------|--------|--------------|----------|
| Zero-Shot LLM        | 53.11%    | 92.78% | 1            | \$2.56   |
| Binary-Search        | 69.46%    | 88.45% | 8.456        | \$16.45  |
| $\alpha$ -tree (n=1) | 85.53%    | 99.44% | 3.4589       | \$7.34   |
| $\alpha$ -tree (n=2) | 86.45%    | 96.34% | 2.3445       | \$5.12   |

**Table 2: Methods for Searching Optimal Allocation Scheme**

我们的方法成本不到 CoT 的一半，且仅为 ToT 成本的十分之一，同时仍能实现可比的性能，突出显示了我们的图结构推理的广泛适用性以及边缘 - 云协作的有效性。

为了评估我们的协作方法是否充分利用本地部署的 SLM 进行推理，我们计算了 SLM 推理时间占总体推理时间成本的比例，以及分配给 SLM 的子任务百分比。这些指标与基线 *DataShunt* 进行了比较，结果如图 5 所示。堆叠柱状图展示了 SLM 和 LLM 的具体数值，而折线图反映了 SLM 的比例。如图所示，在所有基准测试中，我们的分配策略更全面地利用了 SLM，推理时间和任务分配百分比分别平均超出基线 11.99% 和 21.92%。

**图 5: SLM 在时间成本和子任务数量中的比例。**

### 5.3 数据集构建的效率与质量

为了验证我们的方法在数据集构建中对任务难度评估和最优分配搜索的有效性，我们将我们的  $\alpha$ -树与其他基线搜索方法进行了比较。我们建立了两种基线方法：**零样本大语言模型（Zero-shot LLM）**：该方法利用大语言模型评估子任务的难度。评估模型使用 GPT-4o，采用问答格式，将任务内容、所有子任务以及当前子任务纳入其中。

**提示。二分查找：**该方法采用二分查找策略。最初，所有子任务都使用 GPT-4o 统一处理。如果推理结果正确，则将一半子任务随机分配给 Llama 3-8B，这种随机分配重复 N 次直到成功。在第二轮二分查找中，分配给较大模型的剩余子任务的一半再次随机分配给较小模型。该过程持续进行，直到所有子任务要么由较小模型处理，要么推理仍然不正确。我们将 N 设为 5。

对于我们的  $\alpha$ -树方法，该方法根据难度顺序地搜索最优解，我们在 n=1 和 n=2（其中 n 表示每个搜索步骤中更改分配的模型数量）下进行了测试。比较指标包括两个质量评估指标：SLM 比例：最终分配方案中较小的设备模型所占的比例，以及 SR：应用最终分配方案后的推理成功率。此外，我们考虑了两个成本指标：# 评估：每次评估都需要根据分配策略对所有子任务进行完整的推理过程，以及 API 成本，指在数据集构建过程中调用基于云的 LLM API 时产生的费用。

我们在 MATH 基准上可视化了构建数据集的质量和成本。如表 2 所示，就构建质量而言，我们的  $\alpha$ -树在保持最高推理准确率的同时，实现了最高比例的小模型使用。这表明我们的方法找到了更高效的协作分配策略，在最小化推理成本的同时确保高质量的推理。零样本 LLM 导致 SLM 使用比例最低，这表明 LLM 无法评估子任务的难度。二分搜索相比于我们的方法，也表现出较低的 SLM 比例。虽然增加搜索次数 (N) 可能通过随机搜索获得更好的结果，但这种无序且纯随机的策略会产生显著成本。即使在当前的 5 次搜索下，成本也已经是我们方法的数倍。进一步扩展将带来不可持续的成本。自然，LLM 评估的成本最低

### 5.4 消融研究

在这项消融研究中，我们演示了图结构和任务分配机制对我们 DoT 模型的效率和有效性的影响。在移除图结构的情况下，使用顺序推理框架代替。在没有任务分配的情况下，采用单一语言模型或简单的推荐策略。如表 3 所示，完整的 DoT 模型包含这两个组件，在各种任务中都能实现高准确性，同时保持较低的时间和 API 成本。当移除图组件时，准确性和效率均有所下降，这表明图结构对增强推理性能以及基于图的并行执行的有效性具有影响。

此外，移除任务分配机制会导致更高的 API 成本和增加的时间，而准确性并没有显著提升。例如，在 P3 任务中，移除分配机制并仅使用 GPT-4o 的成本为 4.80¢，而 DoT 的成本为 1.58¢，尽管准确性只是略微提高（43% 对 41%）。这表明，分配机制有助于在准确性和效率之间取得平衡。

| Model                                | Logical Reasoning |      |       |      |      |       | Web Browsing |      |        | Solving Math Problems |      |       |       |      |       |      |      |       | Commonsense Reasoning |      |       |
|--------------------------------------|-------------------|------|-------|------|------|-------|--------------|------|--------|-----------------------|------|-------|-------|------|-------|------|------|-------|-----------------------|------|-------|
|                                      | P3                |      |       | SCAN |      |       | WebShop      |      |        | MATH                  |      |       | CHAMP |      |       | DROP |      |       | CSQA                  |      |       |
|                                      | Acc               | Time | Api   | Acc  | Time | Api   | Acc          | Time | Api    | Acc                   | Time | Api   | Acc   | Time | Api   | Acc  | Time | Api   | Acc                   | Time | Api   |
| DoT w/o Graph                        | 38%               | 36.4 | 1.44€ | 57%  | 5.2  | 1.18€ | 31%          | 20.7 | 4.97€  | 56%                   | 20.9 | 0.99€ | 52%   | 27.7 | 0.82€ | 80%  | 4.2  | 0.26€ | 77%                   | 9.6  | 0.48€ |
| DoT w/o Allocation (GPT-4o)          | 43%               | 39.8 | 4.80€ | 64%  | 10.7 | 3.66€ | 39.5%        | 19.4 | 13.89€ | 78%                   | 36.9 | 7.68€ | 64%   | 30.4 | 5.05€ | 84%  | 12.8 | 1.69€ | 85%                   | 20.6 | 1.49€ |
| DoT w/o Allocation (Llama 3-8B)      | 29%               | 21.5 | N/A   | 52%  | 4.2  | N/A   | 3.5%         | 10.7 | N/A    | 49%                   | 19.8 | N/A   | 47%   | 15.8 | N/A   | 77%  | 4.3  | N/A   | 76%                   | 8.9  | N/A   |
| DoT w/o Allocation (Simple Referral) | 33%               | 31.1 | 3.21€ | 57%  | 6.3  | 2.41€ | 12.5%        | 13.5 | 6.27€  | 51%                   | 26.3 | 3.54€ | 53.5% | 19.6 | 2.15€ | 80%  | 6.4  | 0.87€ | 79%                   | 14.6 | 0.64€ |
| DoT (ours)                           | 41%               | 23.5 | 1.58€ | 63%  | 5.5  | 1.20€ | 31%          | 17.2 | 4.97€  | 59%                   | 22.6 | 1.02€ | 58%   | 16.1 | 0.84€ | 85%  | 4.9  | 0.32€ | 82%                   | 9.9  | 0.49€ |

Table 3: Result of ablation study.

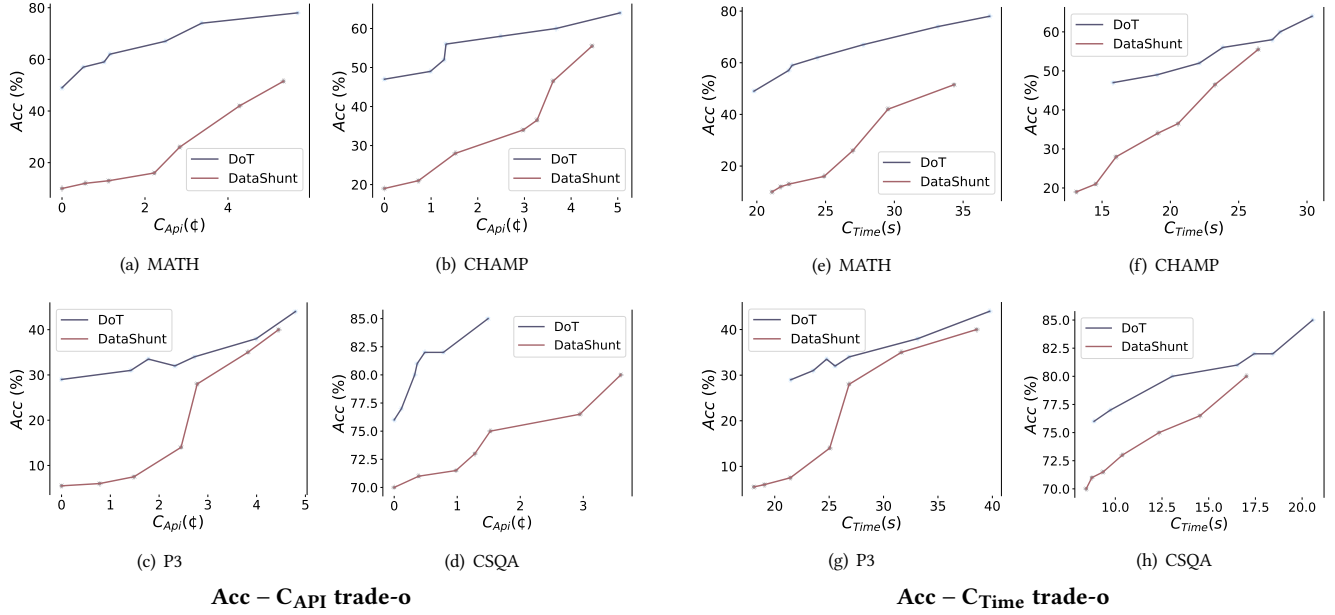


Figure 6: Acc - Cost trade-o curves on 4 benchmarks.

此外，我们惊讶地发现，在不考虑成本的情况下，仅使用基于云的 LLM 可以大大提高在困难的 MATH 和 CHAMP 基准测试上的推理准确性，其中准确率分别从 63% 提升到 78% 以及从 57% 提升到 64%，相比最佳基线。

## 5.5 精度与成本的权衡

消融研究表明，我们的图上推理方法显著增强了大语言模型（LLMs）的推理能力，尽管计算成本更高。在本节中，我们深入探讨成本与推理准确性之间的详细关系，以确定在不同预算约束下可实现的性能上限。基于我们原始的任务分配策略，我们通过根据子任务难度将更多子任务按比例分配给基于云的 LLM 或本地部署的 SLM，生成多样的模型分配方案，从而形成图中所示的各种数据点。我们还引入了 DataShunt 作为基线，其中通过调整 LLM 评估阈值来改变 LLM 处理任务的比例。我们在四个基准数据集上测试了我们的方法，绘制了准确率与 API 成本以及准确率与时间成本的权衡曲线。如图 6 所示，我们的方法曲线始终高于基线曲线。

DoT 的结果显著低于 DataShunt，这表明 DoT 的模型分配方法相当高效。此外，随着预算的增加，我们的 DoT 方法能够达到更高的性能上限。

## 6 结论

总之，我们提出的思想分工协作推理框架通过在本地和云端资源之间智能分配任务，有效提升了边缘设备上大模型的推理性能。DoT 不仅将复杂任务拆分为可管理的子任务，还利用有向无环图优化推理路径，从而实现边缘 - 云的并行处理。此外，即插即用适配器能够准确评估子任务的难度以进行分配，确保高推理性能，同时显著降低时间和金钱成本。展望未来，我们希望在更多、更广泛的场景中应用我们的推理框架，致力于推动边缘人工智能的发展。

## 致谢

本工作部分由中国国家自然科学基金（项目编号 231AA02114 和 62472241）资助，部分由智慧人工智能与清华大学的联合项目资助。

参考文献

[1] Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Harki-rat Behl 等人 . 2024. Phi-3 技术报告：在你的手机上运行的高性能语言模型 . arXiv 预印本 arXiv:2404.14219 (2024).[2] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk 等人 . 2024. 思维图：利用大型语言模型解决复杂问题 . 载于 AAAI 人工智能大会论文集 , 第 38 卷 . 17682–17690.

[3] Dong Chen, Yueting Zhuang, Shuo Zhang, Jinfeng Liu, Su Dong, 和 Siliang Tang. 2024. 数据分流：小模型与大模型协作以降低成本并提升性能 . 载于 AAAI 人工智能大会论文集 , 第 38 卷 . 11249–11257.[4] Lin Chen, Fengli Xu, Nian Li, Zhenyu Han, Meng Wang, Yong Li, 和 Pan Hui. 2024.

安全性是一个正在被广泛研究的主题。[23] 林季、唐佳明、唐昊天、杨尚、陈伟明、王伟辰、肖光轩、党星宇、干创、韩松。2024。AWQ：针对设备上的大语言模型压缩与加速的激活感知权重量化。《机器学习与系统论文集》6 (2024), 87–100。[24] 陆珍言、李翔、蔡东齐、易荣杰、刘方明、张西文、Nicholas D Lane、徐孟伟。2024。小型语言模型：调查、测量与见解。arXiv 预印本 arXiv:2409.15790 (2024)。[25] 毛宇骏、金允、周一伦。2024。CHAMP：用于大语言模型数学推理能力精细分析的竞赛级数据集。arXiv 预印本 arXiv:2401.06961 (2024)。[26] Alex Mari. 2019。语音商务：理解与购物相关的语音助手及其对品牌的影响。(2019)。[27] Meta。2024。介绍 Meta Llama 3：迄今最强大的公开可用大语言模型。https://ai.meta

附录  
补充实验

A.1 适配器的普适性

我们进行了两个补充实验，以评估适配器在不同任务中的泛化性能。**类别内泛化**：在三个数学基准内，训练两个基准并测试剩下的一个。**跨类别泛化**：在六个基准上训练，并测试剩下的一个。我们还评估了适配器在没有任何训练情况下的性能。结果如表 4 所示，表明在数学相关基准内的类别内泛化几乎与针对特定任务的训练一样有效。虽然跨类别应用中性能略有下降，但仍然优于没有训练的情况。

A.2 任务分解质量评估

为了评估任务分解的质量，我们将我们的方法与普通提示策略进行比较，重点评估子任务的独立性以及分解策略对最终推理准确性和推理成本的影响。对于独立性，我们的目标是确定生成的子任务是否独立（即没有重叠的数学计算），并计算独立率以评估性能。我们首先人工标注了 MATH 基准中 50 个任务分解结果的独立性。我们的方法在独立率上达到了 91.3%，显著优于仅为 74.2% 的普通提示策略。这证明了我们任务分解方法的高质量。

随后，我们使用 GPT-4o 来评估两种方法生成的子任务的独立性，并将其结果与人工标注进行比较。GPT-4o 的评估与人工标注达到 93% 的一致性，这表明 GPT-4o 可以提供与人工相当的标注质量，并且可以有效用于大规模评估。因此，我们利用 GPT-4o 对整个测试集的任务拆解独立性进行评估，结果如表 5 所示。结果表明，我们的任务拆解方法在子任务独立性方面比基线方法平均提高了 15.7%，显示出明显优势。此外，这一改进有助于提升推理准确性并降低推理成本。

A.3 子任务数量对推理效率的影响

随着问题变得更加复杂以及分解子任务的数量增加，我们的框架是否能够在保持准确性的同时实现推理效率的显著提升，仍然是一个有价值的研究问题。因此，我们进行了补充实验，在实验中我们将我们的方法与基线方法进行比较，后者是在 SLM 上依次推理所有子任务，并基于不同数量的子任务进行统计分析。表 6 的结果表明，我们的模型在不同数量的子任务节点的任务中始终保持较高的推理效率。

此外，随着子任务数量的增加，效率的提升变得越来越明显。当

| (a) 类内泛化: MATH rai 测试准确                  |       |       |       |       |       |
|------------------------------------------|-------|-------|-------|-------|-------|
| Train                                    |       | API   |       |       |       |
| DRO, CHAMP                               | MATH  | MATH  | 59%   | 22.6  | 1.02¢ |
|                                          | /     | MATH  | 58%   | 26.9  | 1.28¢ |
|                                          | /     | MATH  | 32%   | 33.2  | 2.18¢ |
| (b) 类内泛化: CHAMP ai 测试准确率                 |       |       |       |       |       |
| Train                                    |       | API   |       |       |       |
| MATH, DRO                                | CHAMP | CHAMP | 58%   | 16.1  | 0.84¢ |
|                                          | /     | CHANP | 56.5% | 20.2  | 1.32¢ |
|                                          | /     | CHANP | 27%   | 28.4  | 2.46¢ |
| (c) Intra-category generalization: DRO   |       |       |       |       |       |
| Train                                    | Test  | Acc   | Time  | API   |       |
| DRO                                      | DRO   | 85%   | 4.9   | 0.32¢ |       |
| MATH, CHAMP                              | DRO   | 81.0% | 6.4   | 0.43¢ |       |
| /                                        | DRO   | 72.0% | 7.2   | 0.49¢ |       |
| (d) 跨类别泛化: CSQA                          |       |       |       |       |       |
| Train                                    | Test  | Acc   | Time  | API   |       |
| CSQA                                     | CSQA  | 82%   | 9.9   | 0.49¢ |       |
| DRO, CHAMP, MATH, P3, SCAN, Webshop      | CSQA  | 76%   | 12.5  | 0.58¢ |       |
| /                                        | CSQA  | 70%   | 15.2  | 0.84¢ |       |
| (e) Cross-category generalization: MATH  |       |       |       |       |       |
| Train                                    | Test  | Acc   | Time  | API   |       |
| MATH                                     | MATH  | 59%   | 22.6  | 1.02¢ |       |
| DRO, CHAMP, P3, SCAN, Webshop, CSQA      | MATH  | 53%   | 29.1  | 1.34¢ |       |
| /                                        | MATH  | 32%   | 33.2  | 2.18¢ |       |
| (f) Cross-category generalization: CHAMP |       |       |       |       |       |
| Train                                    | Test  | Acc   | Time  | API   |       |
| CHAMP                                    | CHAMP | 58%   | 16.1  | 0.84¢ |       |
| DRO, MATH, P3, SCAN, Webshop, CSQA       | CHAMP | 50%   | 19.6  | 1.35¢ |       |
| /                                        | CHAMP | 27%   | 28.4  | 2.46¢ |       |
| (g) Cross-category generalization: P3    |       |       |       |       |       |
| Train                                    | Test  | Acc   | Time  | API   |       |
| P3                                       | P3    | 41%   | 23.5  | 1.58¢ |       |
| DRO, CHAMP, MATH, SCAN, Webshop, CSQA    | P3    | 33%   | 29.3  | 1.73¢ |       |
| /                                        | P3    | 15%   | 35.1  | 2.38¢ |       |

Table 4: E-艘的上的 va@at li bili

子任务数量超过八个时，推理时间的减少甚至超过 30%。这一结果本质上是合乎逻辑的，因为随着子任务数量的增加，任务图的复杂性增加，从而扩大了并行推理的潜力，并增强了推理时间方面的优势。

B 基准和实现细节

B.1 数学

- **数学** [14]. 启发式数学能力测试（MATH），包括来自美国著名数学竞赛如 AMC 和 AIME 的 12,500 道题目。这些题目收集自 AoPS 等平台，测试超出标准 K-12 数学的高级问题解决能力。每道题

(a) CSQA 方法上的分解独立性评估 I 依赖性准确率 T

| M                               | n   | n   | n    | ime   | API |
|---------------------------------|-----|-----|------|-------|-----|
| (Ours) DoT decomposition        | 95% | 82% | 9.9  | 0.49¢ |     |
| vanilla prompting decomposition | 87% | 81% | 13.2 | 0.77¢ |     |

(b) 在 MATH 上的分解独立性评估

| Method                          | Independence | Acc | Time | API   |
|---------------------------------|--------------|-----|------|-------|
| (Ours) DoT decomposition        | 90%          | 59% | 22.6 | 1.02¢ |
| vanilla prompting decomposition | 76%          | 57% | 38.4 | 1.78¢ |

(c) P3 方法 I 依赖性累积的分解独立性评估

| M                               | n   | n     | n    | Time  | API |
|---------------------------------|-----|-------|------|-------|-----|
| (Ours) DoT decomposition        | 86% | 41%   | 23.5 | 1.58¢ |     |
| vanilla prompting decomposition | 71% | 36.5% | 24.5 | 2.23¢ |     |

表 5：分解独立性评估

| # Subtask | P3        |            | MATH      |            |
|-----------|-----------|------------|-----------|------------|
|           | ↑Accuracy | ↓Cost_time | ↑Accuracy | ↓Cost_time |
| 3         | 14.5%     | 18.3%      | 9%        | 13.5%      |
| 4         | 18%       | 21.7%      | 13%       | 15.5%      |
| 5         | 20%       | 25.4%      | 14%       | 17.2%      |
| 6         | 13%       | 22.6%      | 17%       | 18.4%      |
| 7         | 14%       | 23.9%      | 15%       | 17.6%      |
| 8         | 12%       | 28.7%      | 13%       | 25.7%      |
| >8        | 17.5%     | 32.1%      | 18%       | 30.3%      |

Table 6: Evaluation of DoT’s Scalability

包括逐步的解决方案和最终答案。数据集涵盖七个学科：初等代数、代数、数论、计数与概率、几何、中级代数和预备微积分，难度级别从 1（简单）到 5（具有挑战性）不等，使模型能够学习和应用各种数学启发式方法。

- **CHAMP** [25]。概念与提示注释数学问题（CHAMP）。该基准包含 270 道非常规、竞赛水平的问题，来源于恩格尔的《问题解决策略》。这些问题涵盖五类：数论、多项式、序列、不等式和组合学，要求运用创造性策略和特定技巧。每道题都包含可最终检查的答案以及自然语言的逐步解答。数据集包含 54 个概念和 330 条提示，每道题平均有 1.4 个概念、1.7 条提示和 6 个解题步骤。题目陈述平均为 20.2 个词，解答步骤平均为每步 10.9 个词，突显了该数据集的复杂性和挑战性。
- **DROP** [9]。段落离散推理（DROP）是一个英文阅读理解数据集，包含 96,000 个对抗性创建的问题。它要求系统在问题中解析引用，并执行离散推理操作，如加法、计数或对段落中多个输入位置进行排序。该数据集由众包创建，并衍生自为复杂问题设计的维基百科段落。与以往的数据集相比，DROP 需要对段落内容有更深入的理解，并通过严格的验证来确保其开发集和测试集的质量。

B.2 逻辑

- **P3** [30]。Python 编程难题 (P3) 引入了一种用于评估程序合成的新型编程挑战。P3 包含 397 个基于 Python 的难题，目标是找到一个输入，使给定的函数返回 True。这些难题涵盖了各种难度等级，从简单的字符串操作到复杂的算法问题。
- **扫描** [20]。这个基准用于评估序列到序列学习能力，将简化的自然语言命令翻译为动作序列。SCAN 包含由短语句法生成的 20,910 条命令，这些命令描述了基本动作，例如“跳跃”或“行走”，以及它们的组合变化（例如，“向左跳动”）。翻译过程中涉及的逻辑结构和约束使 SCAN 成为评估大型语言模型推理能力的理想工具。

B.3 常识

- **COMMONSENSEQA** [36]。这个基准是一个具有挑战性的数据集，旨在测试常识问答能力。它由 12,247 个多项选择题组成，这些题目是利用 CONCEPTNET 的概念创建的。每个问题都是由众包工作人员撰写的，目的是在多个与源概念有语义关系的目标概念之间进行区分，从而鼓励使用先验知识和复杂推理。

B.4 设备端 AI 助手应用

- **网络商店** [42]。WebShop 作为一个模拟的电子商务环境，提供了 118 万种真实世界的产品和 12,087 条众包文本指令。它旨在评估语言代理，挑战代理在不同网页上导航并根据自然语言产品规格执行操作的能力。代理会遇到各种障碍，例如解释组合指令、重新制定查询、理解网页上的噪声文本，同时还需策略性地探索以满足产品需求。WebShop 的模块化设计将网站导航与特定任务元素分离，使其能够轻松适应新的任务和领域。该数据集提供了一个强有力的平台，用于评估语言代理在互动的、受真实世界启发的环境中的能力，强调其理解和执行复杂指令的能力。

B.5 各基准的实现细节

**数学。**对于数学任务，我们采用一个结构化的工作流程，包括四个关键阶段：任务分解、模型分配、依赖图构建，以及基于图的逐步推理。在任务分解阶段，我们通过手动分解的复杂问题示例来提示大语言模型（LLM）。然后指示 LLM 生成可管理的子任务，这些子任务共同解决主要问题。随后，我们让 LLM 建立子任务之间的依赖关系，其中关系表示  $Step_i$  必须在  $Step_j$  之前完成。利用得出的依赖关系，我们通过广度优先搜索（BFS）构建推理图。具有相同深度的子任务会并行处理。在所有子任务完成后，我们进行最终查询。

以获得最终解决方案。该解决方案通过与实际情况的比较, 进行基于大型语言模型 ( LLM ) 的评估。

**CHAMP.** 对于 CHAMP 任务, 这个过程在很大程度上与 MATH 过程类似, 主要区别在于提供了少量人类编写的分解示例。

**DROP.** 对于 DROP 任务, 我们调整了方法以适应基于给定文本的问题格式。每一步的提示都结合了数据集中提供的相关背景信息, 同时保持在其他基准测试中使用的核心实现结构。

**P3.** 编程难题任务提出了独特的挑战, 要求大型语言模型 ( LLM ) 生成能够使给定函数输出 “True” 的输入。除了难题描述之外, 我们还向 LLM 提供了最终输入的预期数据类型。P3 的评估过程不同于其他基准测试: 我们使用 LLM 生成的输入执行程序, 并根据程序的输出评估正确性。

**扫描。**对于专注于将自然语言指令翻译成动作序列的 SCAN 任务, 直到评估的步骤保持不变。最后阶段包括使用少量示例将模型的自然语言输出转换为标准化的动作序列。评估通过将这些序列与真实答案直接比较进行, 以确保输出准确匹配预期的动作。

**COMMONSENSEQA.**对于 CSQA 任务, 其形式为基于常识的多项选择题, 我们的实现结构与其他任务相同, 同时将问题及其选项提供给大型语言模型 ( LLM ) 以进行后续的任务分解和推理。最终评估由 LLM 从提供的选项中选择最合理的答案。所选答案将进行清理, 并与正确答案直接比较。

**网络商店 ( WebShop ) 。**由于其互动性, WebShop 任务呈现出独特的挑战, 每一个操作都会影响后续的状态和可用选项。与我们的其他实验不同, WebShop 并不适合构建完整的、预先定义的推理图。相反, 我们的框架根据购物会话的当前状态动态生成并执行子任务, 在其中购物过程被划分为高层次的子任务 / 思路。我们在生成动作时为每一步使用了模型分配, 可以是 ‘思考 ( think[] ) ’, ‘点击 ( click[] ) ’ 或 ‘计划 ( plan[] ) ’。第一步是根据指令生成一个在网站上搜索的关键词。然后, 将排名前  $N = 10$  的匹配商品记录下来以用于后续流程。在网页对话的基础上, 我们实施了任务分解过程, 以获得后续行动的路线图。每一步同时提示给大型语言模型 ( LLMs ), 以获取详细信息关于

## C 提示

### C.1 任务分解

我现在将给你一个 [ 基于问题类型 ] 的问题。问题的类型是 type。请将此问题分解为几个易于解决的步骤。

1 个例子如下: [ 手写示例 ]

现在命令是一个问题, 请像例子一样将其分解为易于解决的步骤。回答格式: ( 请将每个分解的问题步骤分别写在一行上, 以数字开头。)

要解决问题 “xxx”, 我们需要知道: 1. 问题  $step_1$ , 2. 问题  $step_2$ , 3. 问题  $step_3$ 。...

### C.2 依存结构

系统提示:  
现在有一个问题, 我们已经将其分解成许多子问题。我希望你理解这些子问题之间的联系

用户提示:  
初始化问题是主要问题。子问题是步骤。请提供你对这些子问题之间关系的理解。你的回答必须简明扼要。

现在我们需要为这些子问题之间的关系创建标准化连接。现在, 给定问题的以下子任务: [ 问题 ], 确定它们之间的依赖关系:

[ 步骤列表 ]

请以 ‘子问题 A [xxx]-> 子问题 B [xxx]’ 的格式列出依赖关系, 表示子问题 A 必须在子问题 B 开始之前完成。请从逻辑的角度识别任何潜在的条件依赖关系。

答案格式: ( 请严格遵循此格式。每个依赖项应换行分隔。无需解释。 ) 步骤  $ID_i$  [ 子问题 i ] -> 步骤  $ID_j$  [ 子问题 j ] 步骤  $ID_j$  [ 子问题 m ] -> 步骤  $ID_n$  [ 子问题 n ] ...

### C.3 子任务推理

系统提示:  
这里有一个数学应用题。我将先提供题目的段落来设置背景。然后, 我会提出一个具体问题, 需要你使用题目描述中的信息, 结合计算和推理来解决它。

段落: [ 段落 ] 问题: [ 问题 ]

我已经将这个数学题分解成几个较小的问题。我会一个接一个地分配子问题, 并提供前一个子问题的结果作为你推理的参考。请根据数学逻辑来解题。

对于每一步到目前为止, 已解决的子问题的答案如下: 格式为 子问题编号 : xxx; 子问题 : xxx; 答案 : xxx。子问题编号 : [ 对应的 ID ]; 子问题 : [ 对应的步骤 ]; 答案 : [ 对应该步骤的解决方案 ]

其中, 子问题的前置问题与此子问题直接相关, 因此请特别关注它们。目前需要解决的子问题是 xxx: {subtask} 基于上述信息, 请提供简明清晰的答案